

Merge Sort

10 April 2026 18:51

Merge Sort is a sorting algorithm based on Divide and Conquer technique.

It works by:

- Dividing the array into two halves.
- Recursively sorting both halves.
- Merging the sorted halves into one sorted array.

Merge Sort is very efficient for large datasets because it has the time complexity of $O(n \log n)$. Close to linear.

How merge sort Works?

0	1	2	3	4	5	6	7	8
38	27	43	3	9	82	10	11	5

$$9/2 = 4$$

Step 1: Divide the array

The array is repeatedly divided into two halves until each subarray contains only one element:

$$4/2 = 2$$

0	1	2	3	4	5	6	7	8
38	27	43	3	9	82	10	11	5
0	1	2	3	4	5	6	7	8
38	27	43	3	9	82	10	11	5
0	1	2	3	4	5	6	7	8
38	27	43	3	9	82	10	11	5
0	1	2	3	4	5	6	7	8
38	27	43	3	9	82	10	11	5

$$mid = (lb + ub) / 2$$

Step 2: Merge Sorted Subarrays

Now single element arrays are merged in sorted order.

0	1	2	3	4	5	6	7	8
38	27	43	3	9	82	10	11	5
0	1	2	3	4	5	6	7	8
27	38	3	43	9	82	10	11	5
0	1	2	3	4	5	6	7	8
3	27	38	43	9	10	11	82	5
0	1	2	3	4	5	6	7	8
3	9	11	11	27	38	43	82	5
0	1	2	3	4	5	6	7	8
3	5	9	11	27	38	43	82	

0	1	2	3	4	5	6	7
3	5	9	11	27	38	43	82

Recursive nature of Merge Sort

Merge sort uses recursion.

A recursive function is a function that calls itself until a stopping condition is reached.

In merge sort:

Base condition: If the array contains only one element, it is already sorted.

Recursive Step: Divide the array into halves and sort both halves recursively.

```
package DivideAndConquer;
public class MergeSortAlgo {
    public static void mergeSort(int arr[], int lb, int ub){
        if(lb < ub){
            int mid = (lb+ub)/2;
            mergeSort(arr, lb, mid);
            mergeSort(arr, mid+1, ub);
            merge(arr, lb, mid, ub);
        }
    }
    public static void merge(int[] arr, int lb, int mid, int ub){
        int n1 = mid - lb + 1;
        int n2 = ub - mid;
        int[] leftArray = new int[n1];
        int[] rightArray = new int[n2];
        for (int i = 0; i < leftArray.length; i++) {
            leftArray[i] = arr[lb + i];
        }
        for (int i = 0; i < rightArray.length; i++) {
            rightArray[i] = arr[mid + 1 + i];
        }
        int i = 0, j = 0;
        int k = lb;
        while(i < n1 && j < n2 ){
            if(leftArray[i] <= rightArray[j]){
                arr[k] = leftArray[i];
                i++;
            }
            else{
                arr[k] = rightArray[j];
                j++;
            }
            k++;
        }
        while(i < n1){
            arr[k] = leftArray[i];
            i++;
            k++;
        }
        while (j < n2) {
            arr[k] = rightArray[j];
            j++;
        }
    }
}
```

```

        k++;
    }
}
public static void display(int[] a, String msg){
    System.out.println(msg);
    for (int i : a) {
        System.out.print(i + " ");
    }
    System.out.println();
}
public static void main(String[] args) {
    int arr[] = {38, 27, 43, 3, 9, 82, 10, 11, 5};
    display(arr, "Before sorting: ");
    mergeSort(arr, 0, arr.length-1);
    display(arr, "After sorting: ");
}
}

```

Output:

```

Before sorting:
38 27 43 3 9 82 10 11 5
After sorting:
3 5 9 10 11 27 38 43 82

```

Homework: Do the dry-run diagram of the code.